Agentic AI · Week 4 — collaborative writing sheet

Friday Oct 9 · the Overleaf hour · everything else is on the main follow-along sheet

One Overleaf project. Everyone writes a section and brings their own references. Nobody's file touches anybody else's, so nothing you do can break the document for the room — and at the end you watch it assemble into one paper with your name in it.

WHAT YOU NEED BEFORE THIS HOUR

- An **Overleaf account via MSU SSO** — overleaf.com/edu/msu. Your own, not the instructor's.
- **Your own Git token.** Account Settings → *Git integration* → Generate. Copy it; it is shown once.
- A terminal, Claude Code, and the Demo 10 package unzipped.

Never use somebody else's token. A token is **per person** and opens *every project that person owns* — their grants, their manuscripts under review. Overleaf's own wording: *"You should never share your authentication tokens."* Yours opens only what you can already see.

1 · JOIN AND CLONE

Open the edit link the instructor puts on the screen. Copy **all** of it, including the #... on the end — the fragment is part of the link.

```
git config --global credential.helper osxkeychain
cd ~/research/writing/tex
git clone https://git@git.overleaf.com/<project-id> classpaper
cd classpaper
```

Password prompt = **your token**, not your SSO password. The keychain remembers it.

```
ls groups/
```

Pick your filename and use it for everything. Your surname, or your NetID if two of you share one:

```
groups/yourname.tex your writing
groups/yourname.bib your references
```

Neither exists yet. You create both. **Two people choosing the same name is now the only way to collide**, so say yours out loud if you are unsure.

Your filename: _____

2 · WRITE, WITH CLAUDE CODE

```
claude
```

Then, in your own words — a real question, not an exercise:

```
Create groups/yourname.tex. I want 150 words on the
agent system I would build for my own research: what
it would read, what it would be forbidden to touch,
and how I would know it was wrong. Start with
. Body text only,
no preamble.
```

"Body text only, no preamble." Say it. Your file gets pasted inside a document that already has its own \documentclass — a second one makes the whole thing fail to build.

Adding a reference? Your own .bib, cited normally:

```
Add 10.1038/s41598-023-49831-6 to
groups/yourname.bib
from my Zotero library, then cite it in my
section.
```

3 · CHECK YOUR OWN FILE FIRST

```
../../../tools/tex-solo.sh groups/yourname.tex
```

This checks **just your section**. With LaTeX installed it compiles it, wrapped in a minimal document with your own .bib. **Without LaTeX it runs text checks instead** — same three defects, no install needed. Ten seconds either way, and it is the difference between contributing and contributing a problem:

```
ok groups/yourname.tex compiles on its own
ok every \cite resolves against your own .bib
ok body text only — no preamble
Safe to push.
```

Three things it catches that would otherwise surface as **somebody else's** problem at the end:

- LaTeX that does not compile — an unclosed \begin{itemize} is the usual one
- a \cite key that is not in your .bib. **LaTeX does not treat this as an error** — it exits 0 and prints ? where your citation should be
- a pasted-in \documentclass or \begin{document}

You do not need LaTeX for this. Without it the script says so and runs the text checks anyway. It cannot then prove your page typesets — that happens at the assembly.

Do not push until this passes. Nothing else in the hour depends on you being careful; this one does.

4 · PUSH

```
../../../tools/overleaf-sync.sh
groups/yourname.tex "my agent system"
```

It commits your two files and pushes. Two things plain git push would not do:

It refuses to push files that are not yours. Edited something outside your own two files and it stops, names the file, and nothing leaves your machine.

It loses races politely. Thirty people pushing to one branch means your push will often be rejected — *not* an error, just somebody arriving first:

```
note someone pushed first (attempt 1/8) —
rebasing onto their work
ok pushed on attempt 2
```

Because your files are yours alone, the rebase never conflicts. **That is the merge happening automatically, and it is the thing worth watching.**

Reload Overleaf. Your file is in groups/. It is not in the document yet, and that is deliberate — see the next section.

5 · WHY NOTHING COMPILES YET

main.tex does not mention your file. Nobody's. So **nothing you push can break the document**, because there is no document yet — only a folder of contributions.

The alternative, where everyone writes straight into a live main.tex, fails badly in a room this size: one unclosed brace, or one duplicate reference, and **every citation in the shared paper renders [?]** for everybody, on the projector, five minutes before the end.

So: contributions now, assembly in front of you.

The instructor will assemble a few times during the hour, not only at the end. Each time, main.tex is regenerated and pushed — so **git pull and reload Overleaf, and the document has grown**, with your section in it. That is your compile: you do not need LaTeX on your own machine to see the typeset result.

6 • THE ASSEMBLY — WATCH THIS PART

The instructor runs:
`tools/tex-assemble.sh`
It lists every contribution with a word count, flags any that still carry a preamble, and reports **every cite key that two different people used**:
`clash choiMachineLearningModels2023 in amy AND bob`
That clash is not a mistake either of them made. You both cited the same paper, and Better BibTeX gave it the same key in both files — which is correct behaviour. But two copies of one entry makes bibtex fail and turns *every* citation in the document into [?].
The fix is to delete the entry from one file. **Both of you keep citing it** — the bibliography is shared, so one copy serves everyone, and you both get [1].

`tools/tex-assemble.sh --write`
regenerates main.tex with every section and every .bib, compiles, and reports the page count. **One paper, everyone's name in it.**

7 • KEEPING THE REVISION HISTORY

Three layers, all free.
Your own edits:

```
git log --oneline -- groups/yourname.tex
git diff HEAD~1 -- groups/yourname.tex
```

Who contributed what:

```
git log --format='%an %s' --no-merges | head -30
git shortlog -sn --no-merges
```

The whole document over time — Overleaf keeps its own full history (an MSU premium feature). *Menu → History*. Every push is a version with a name on it, restorable from the browser without touching git.
Before a second round of edits: git pull. Always. Somebody has pushed since you cloned.

8 • IF SOMETHING GOES WRONG

- **fatal: No configured push destination** — you are one folder too high. cd into the clone; check with git remote -v
- **Authentication failed** — you used your SSO password. It wants the **token**
- **refusing to push changes outside …** — you edited a file that is not yours. git checkout -- FILENAME
- **Your LaTeX does not compile** — from tex-solo.sh, before you pushed. Exactly what it is for. Fix and re-run
- **could not push after 8 attempts** — the project is busy. Wait and run it again. **Your commit is safe locally**
- **nothing works — paste your section into the Overleaf browser editor** as groups/yourname.tex. Ten seconds, always works, and you have not missed the point

Nothing you do can lose anyone's work. Every version is in git and in Overleaf's history, and both scripts stop rather than leaving you half-merged.

WHAT THIS IS ACTUALLY DEMONSTRATING

Not "Claude Code can use git."
Conflicts are a design property, not bad luck. Thirty people writing into one main.tex collide constantly. The same thirty writing into groups/<name>.tex never collide at all — and nobody had to be careful, because the arrangement removed the mistake. That is Week 3's deny list and the subagent with no Write tool, applied to people instead of agents.

And where you cannot design it away, you check. Two people citing the same paper is not a mistake and no folder layout prevents it. So it gets a check that runs *before* the damage rather than after — which is the same reason tex.sh fails on an undefined citation that latexmk is happy to let through.